CS3021 Introduction to Data Structures and Intermediate Programming

AVL Tree: A Height-Balanced BST

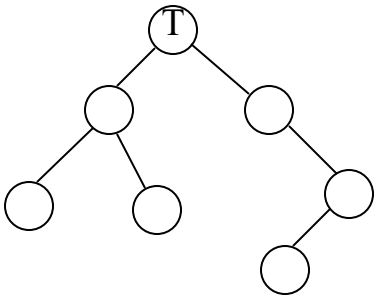
AVL Tree

- A regular *BST* can degenerate into a *singly linked list* after a sequence of insertion and deletion operations, if not done properly.
- An *AVL tree* is a *balanced* binary search tree.
- A *BST* has average performance *O(logN)*, but worst case performance *O(N).*
- An *AVL tree* has *O(logN)* performance of both average and worst cases (since it is balanced).
- Named for inventors G.M. Adelson-Velsky & E.M. Landis.

When is a BST Balanced?

- The *height* of a BST is the number of nodes from the root node to the farthest leaf node.

T.height == 3



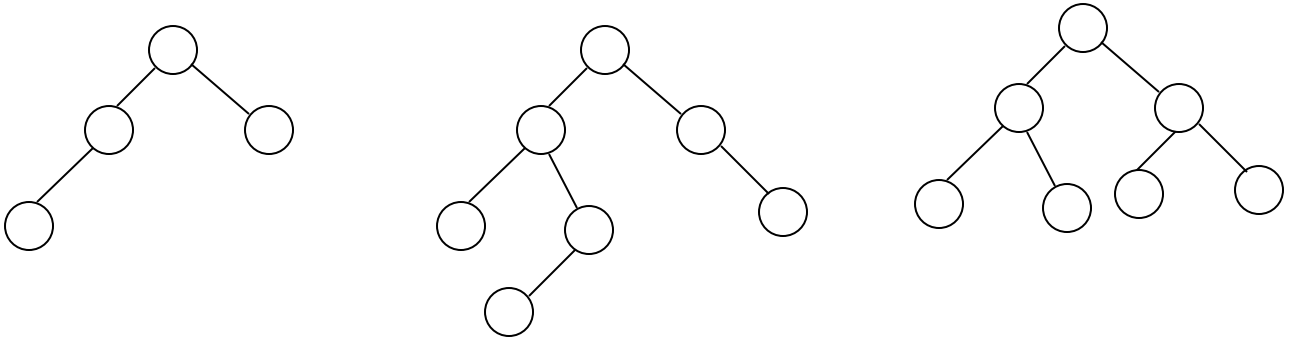
We can also consider the left and right subtree *heights*

- A BST is *balanced* if, for <u>*every*</u> node *N*:

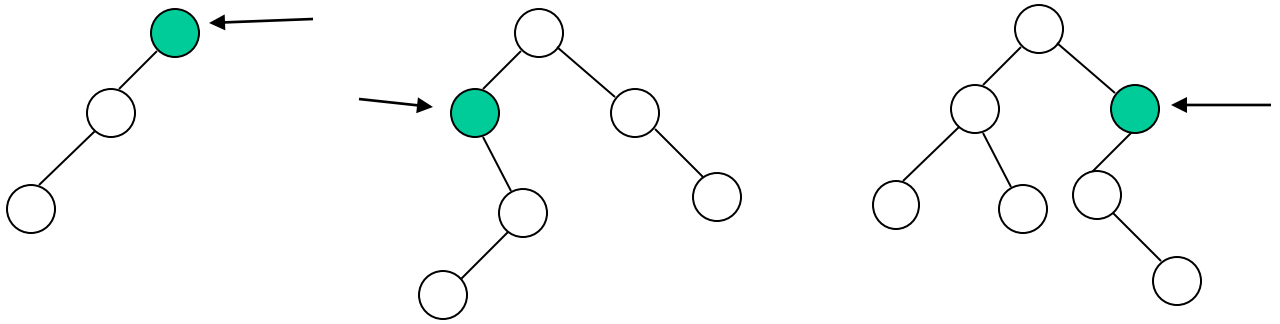
  $|h_L - h_R| <= 1$, where h_L and h_R are the heights of the left and right subtrees of *N*, respectively.

Sample Balanced and Unbalanced BST

Balanced



Unbalanced



AVL – A Balanced BST

- We call a height-balanced BST an *AVL tree.*
- Generally, a balanced BST becomes *unbalanced* after an *insertion* or *deletion.*
- When a BST becomes *unbalanced*, we restructure the tree to make it balanced again.
- Balancing operations are called *rotations*; there are four types of rotations (more about these in a moment).

Balance Factor

- We associate a *balance factor* to every node in an AVL tree.
- The *balance factor* of a node *X* is defined as:

$$X.bf = h_R - h_L$$

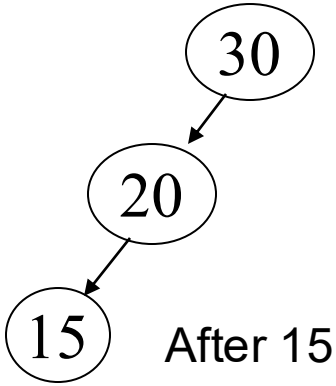
where $X.bf = -1, 0, 1$ for AVL tree

- If the *balance factor* of a node becomes +2 (“heavy” on the right) or -2 (“heavy” on the left) after an insertion or deletion, the tree must be balance using one of the four rotations.

Four Rotations for Balancing

- Single Rotations
  - *LL*: the left subtree of the left subtree of an unbalanced node.
  - *RR*: the right subtree of the right subtree of an unbalanced node.
- Double Rotations
  - *LR*: the right subtree of the left subtree of an unbalanced node.
  - *RL*: the left subtree of the right subtree of an unbalanced node.

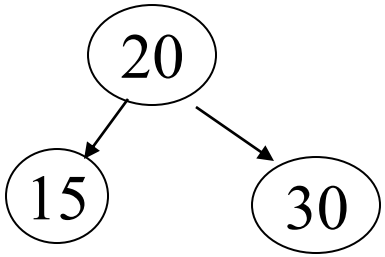
Example LL Rotations

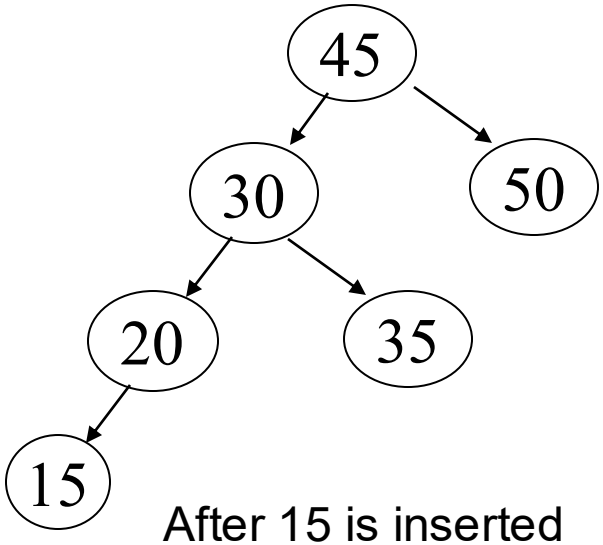


After 15 is inserted

bf(30) == -2

LL rotation at pivot point 20

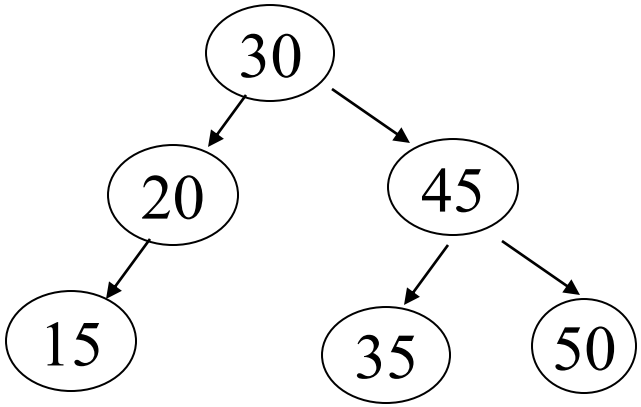




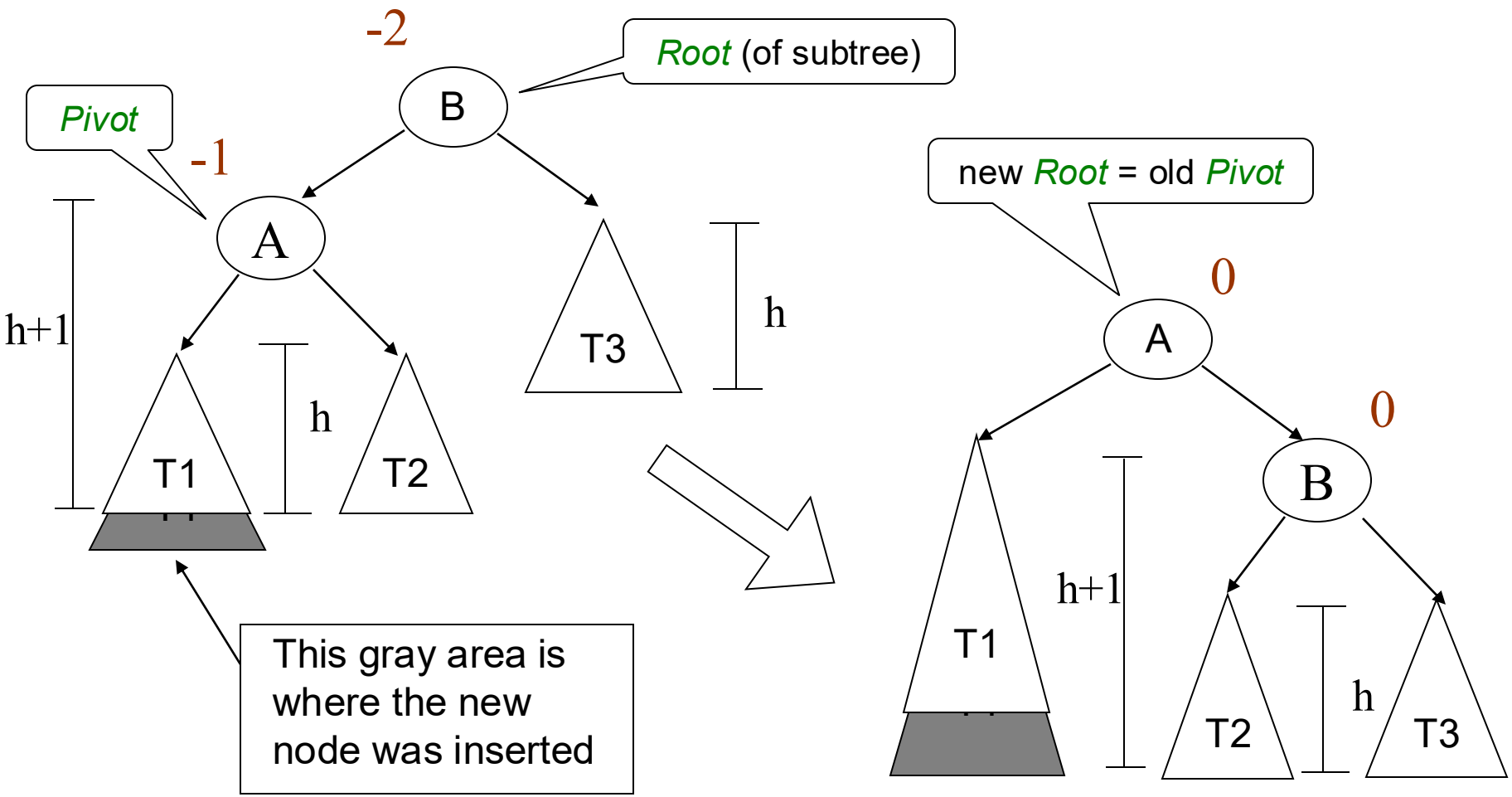
After 15 is inserted

bf(45) == -2

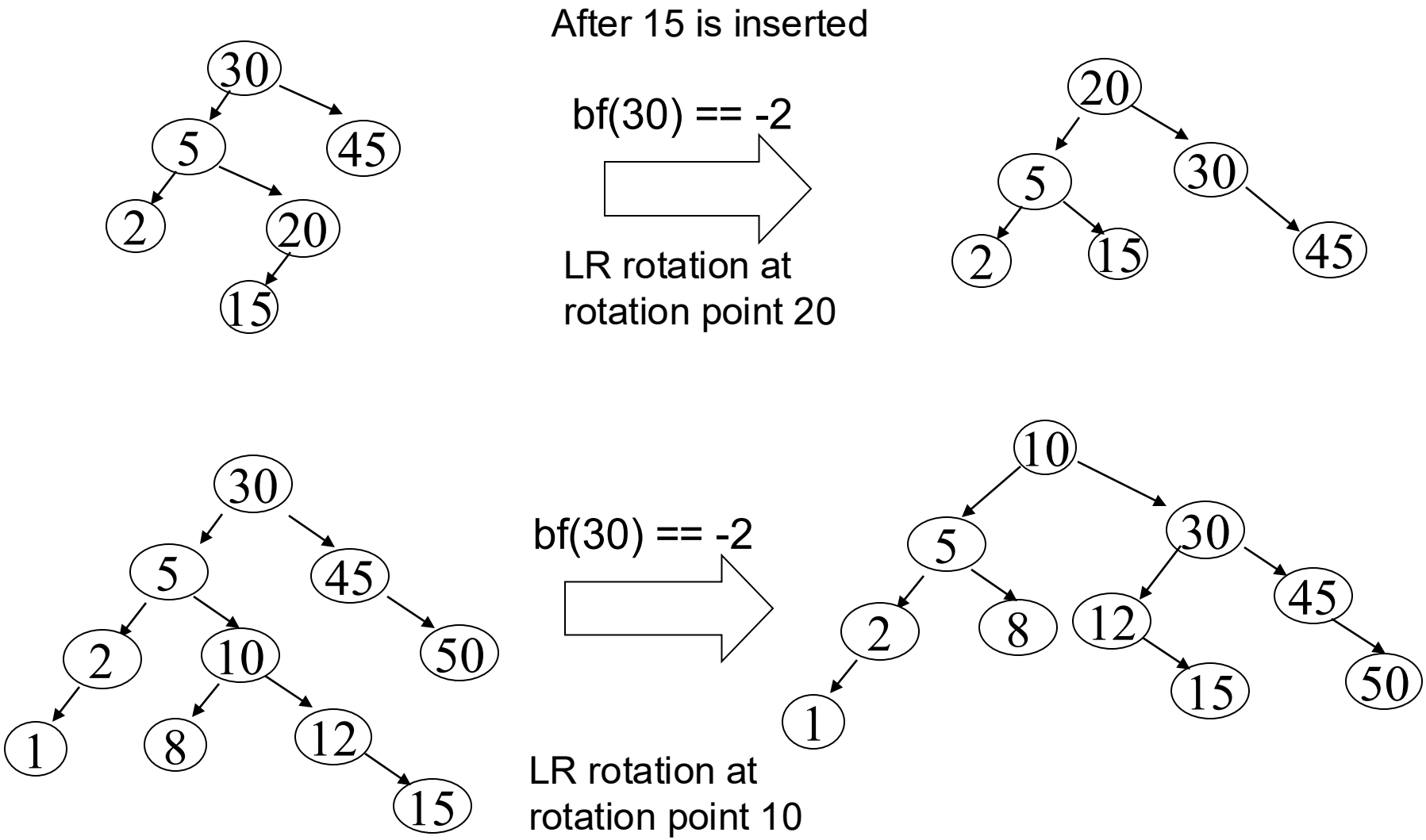
LL rotation at pivot point 30



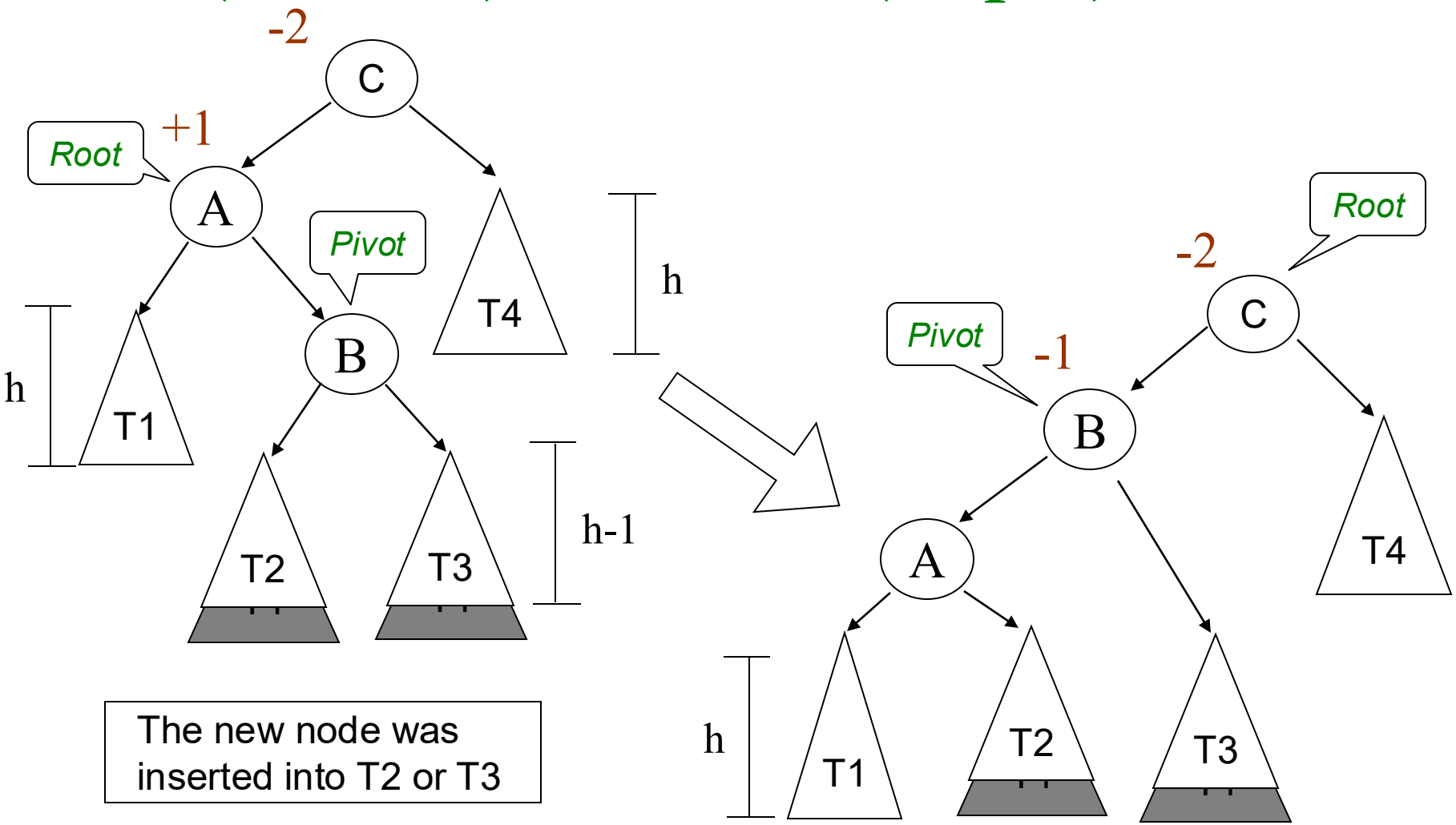
LL (Single) Rotation



Example LR Rotations

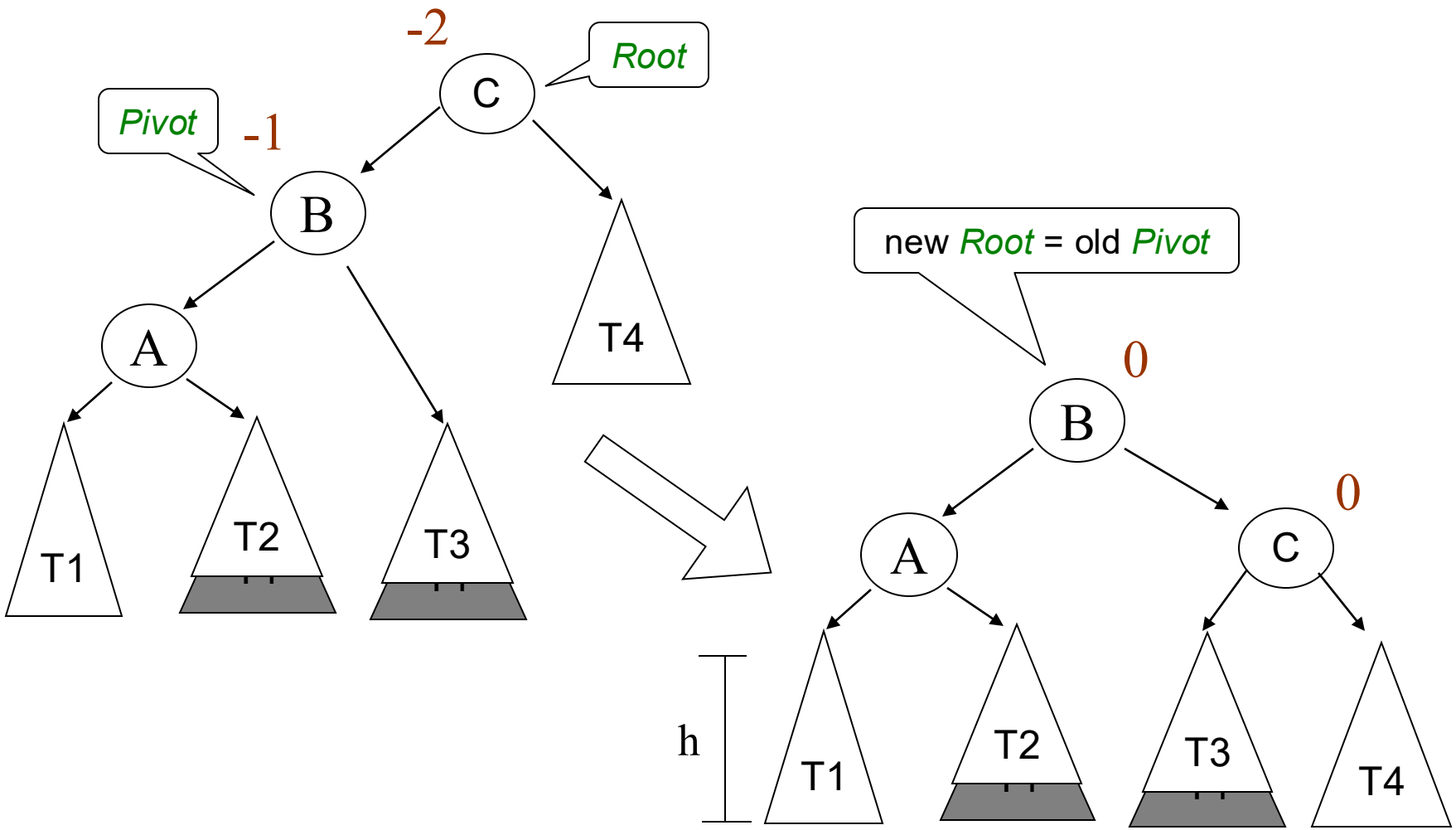


LR (Double) Rotation (step 1)



The new node was inserted into T2 or T3

LR (Double) Rotation (step 2)



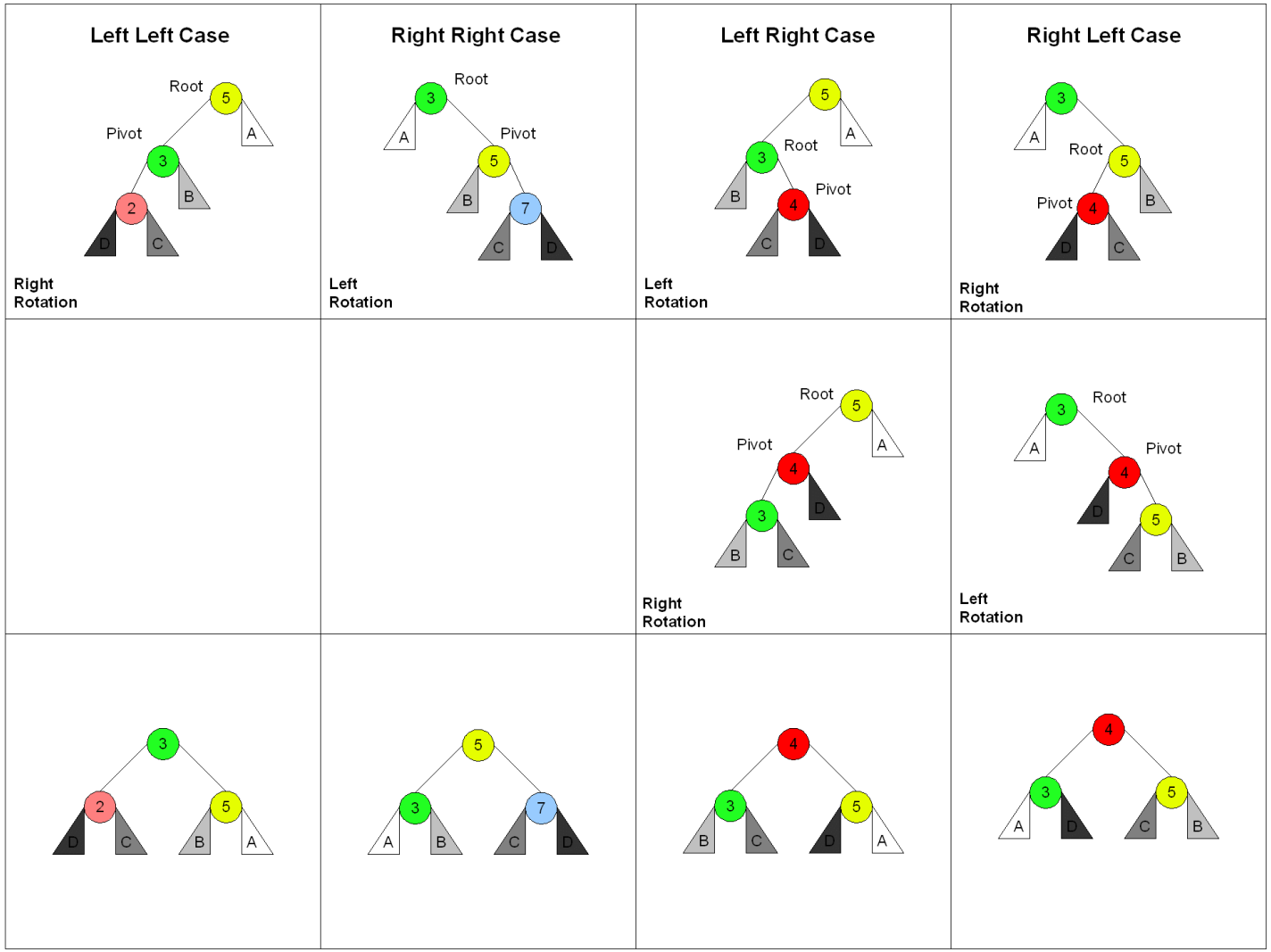
RR and RL Rotations

- RR is exactly symmetrical to LL.
- RL is exactly symmetrical to LR.
- In other words, you can flip the slides for LL and LR, to get RR and RL rotations, respectively.

Rotations Graphically

Initial State

Final State

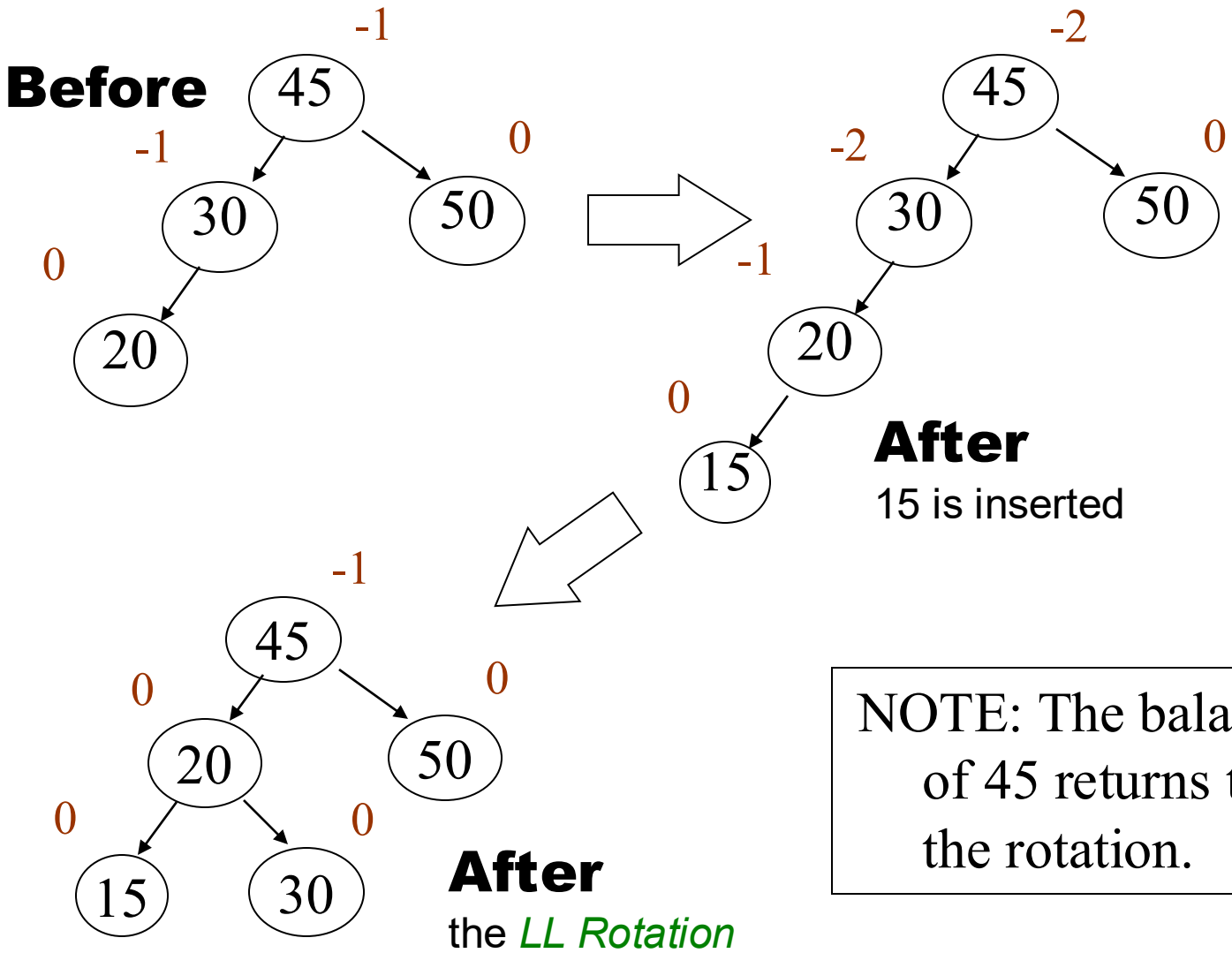


Source: Wikipedia.com

AVL Tree Insertion

- After an insertion, many nodes in the path to the insertion point can become unbalanced.
- We use the unbalanced node *closest* to the insertion point as a rotation pivot – we call this node `pivot`.
- By comparing the values of `pivot`, pivot's `child`, and pivot's `grandchild`, we can determine the rotation type.
- After <u>*one*</u> rotation, all nodes become balanced.

Insertion Example



15
20
30
45

NOTE: The balance factor of 45 returns to (-1) after the rotation.

Exercise: AVL Insertions

- Insert the following two sequences of values into an empty AVL tree:
  1. 3, 5, 11, 8, 4, 1, 12, 7, 2, 6, 10, 9 (RR, LL, and LR rotations).
  2. 12, 1, 4, 3, 7, 8, 10, 2, 11, 5, 6, 9 (LR, LR, RL, LR, LR, and RR rotations).

AVL Tree Implementation

- Declare `AVLTreeNode` which is a subclass of `BinaryTreeNode`
  - The class adds an additional data member called `bf` for storing the balance factor of each node.
- Declare `AVLTree` which is a subclass of `BinarySearchTree`
  - The class overrides the inherited `insert` and `delete` methods.
- Declare constants `LEFT_TALL` (= -1), `EQUAL_HT` (= 0), and `RIGHT_TALL` (= 1).

AVL Tree Insert

```
//public method of the AVLTree class
//Inserts the node newNode; assume no duplicate
void insert(AVLTreeNode newNode) {
   newNode.bf = EQUAL_HT;

   //S contains nodes in the path from the root
   //to the insertion point, including newNode
   Stack S = buildStack(newNode);

   AVLTreeNode child = S.pop();
   AVLTreeNode pivot = S.pop();
   AVLTreeNode grandchild = null;
   boolean     done  = false;

   //continue to next page...
```

AVL Tree Insert Case 1

```
while (!done) {
   //Case 1: pivot was perfectly balanced before
   //        insertion, gets taller on left or
   //        right subtree after insertion
   if(pivot.bf == EQUAL_HT) {
      if(child == pivot.right) //adjust the pivot bf
         pivot.bf = TALL_RIGHT;
      else
         pivot.bf = TALL_LEFT;

      if(S.isEmpty())
         done = true;
      else {
         grandchild = child;
         child = pivot;
         pivot = S.pop();
      }
   } //proceed to Case 2
```

AVL Tree Insert Case 2

```
//Case 2: the newNode gets inserted into the
//        shorter subtree of pivot. The pivot
//        node becomes perfectly balanced
else if ((pivot.bf == TALL_LEFT
            && child == pivot.right) ||
           (pivot.bf == TALL_RIGHT
            && child == pivot.left)){

   pivot.bf = EQUAL_HT;
   done = true;
} //proceed to Case 3
```

AVL Tree Insert Case 3

```
    //Case 3: the newNode gets inserted into the
    //        taller subtree of pivot. We need to
    //        rotate at the pivot node.
    else {
        rotate(pivot, child, grandchild);
        done = true;
        //after the rotation, pivot becomes
        //perfectly balanced and the bf for
        //all nodes from the root to the parent
        //of pivot are the same as their value
        //before the insertion, so we can stop
        //after one rotation
        }
    } //end outer while-loop
} //end insert method
```

Rotate Method

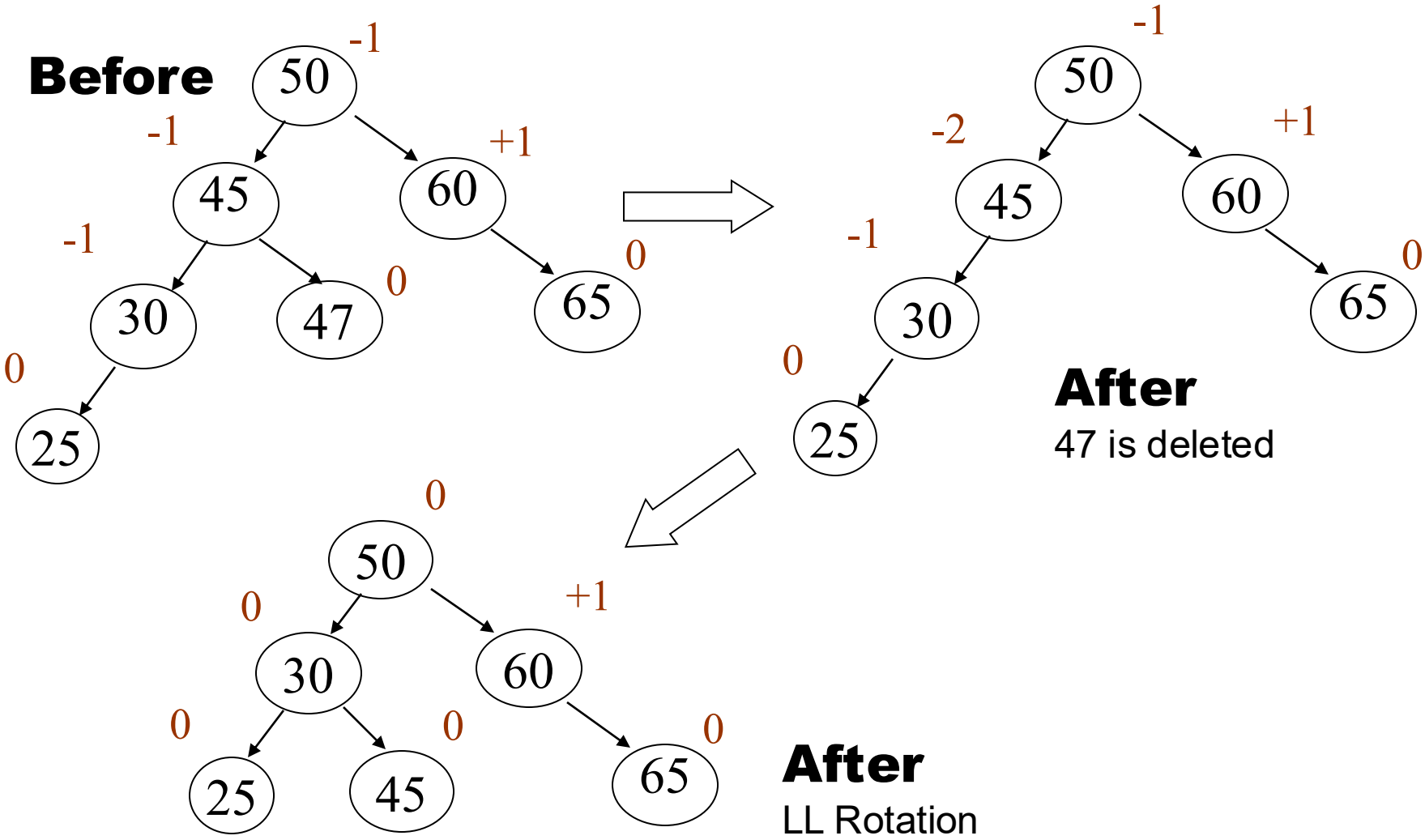
```
private void rotate(pivot, child, grandchild) {

   if(child == pivot.right)
      if(grandchild == child.right)
         rotateRR(pivot, child); //Exer
      else
         rotateRL(pivot, child, grandchild); //Exer
   else              //L
      if(grandchild == child.left)
         rotateLL(pivot, child); //Exer
      else
         rotateLR(pivot, child, grandchild); //Exer

} //end rotate
```

- Note that the four rotate methods update pivot and child bf values

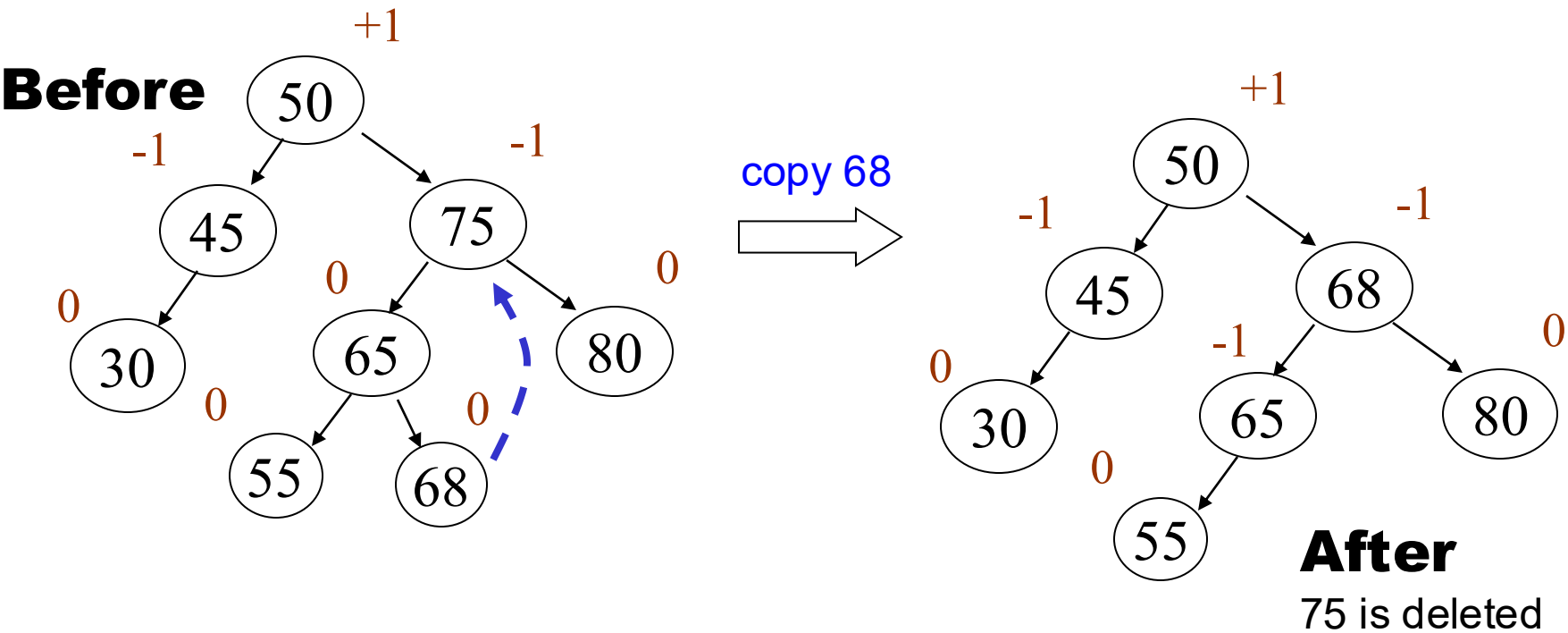
AVL Tree Delete

- When the height of an AVL tree is *O(logN)*, we may need to rotate up to *O(logN)* times after a deletion.
- For rotations, we can apply the same rotations we used in the insertion routine, but the adjustment of balance factors is slightly different.
- Once again, we will use a Stack in the deletion routine to keep track of a path from the root to the deletion point.

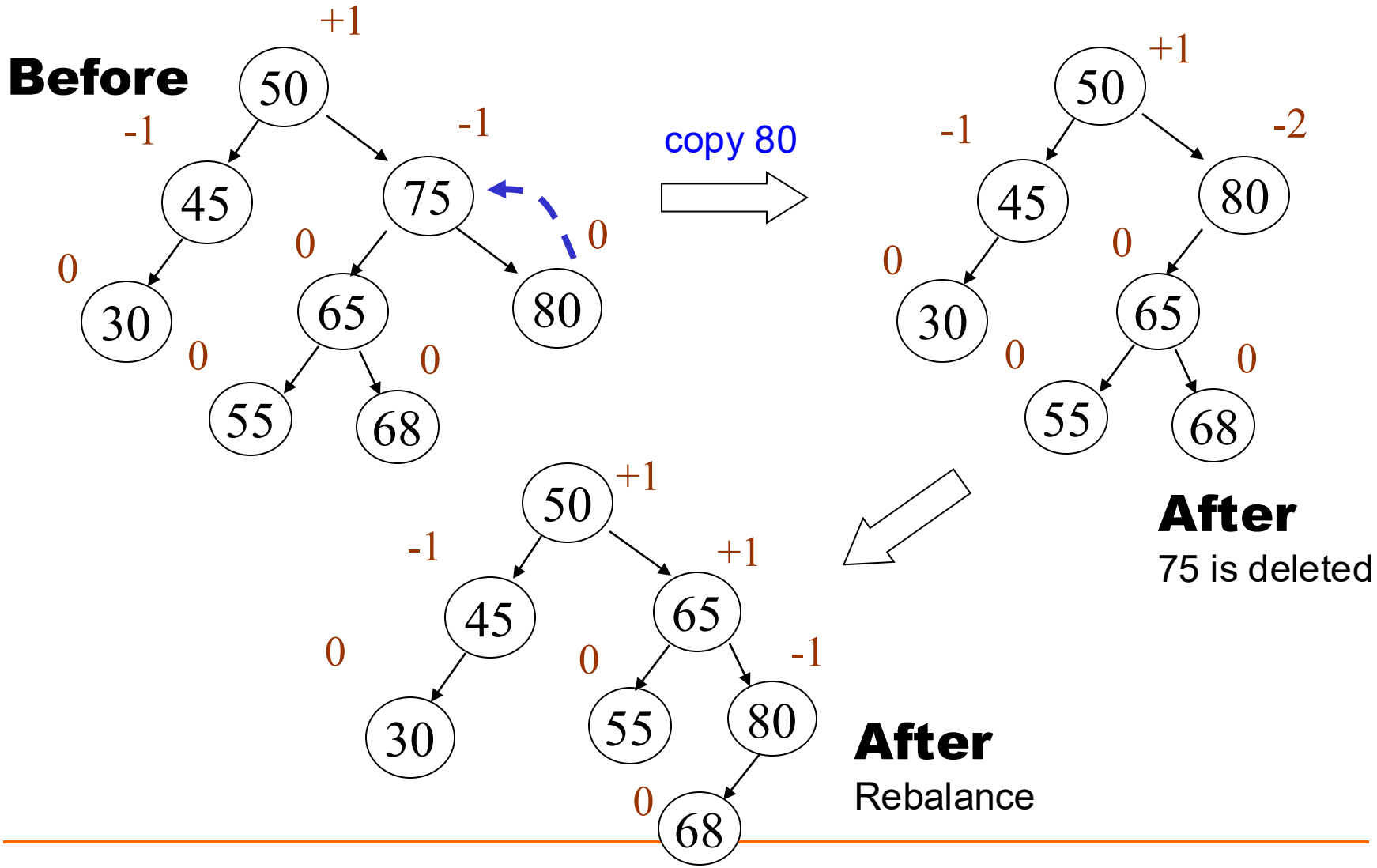
Deletion Example – Leaf Node



Deletion Example - Internal Node (I)



Deletion Example - Internal Node (II)



AVL Tree Delete

```
//public method of the AVLTree class
//Deletes the node with searchKey value
void delete(KeyData searchKey) {
   AVLTreeNode delNode = search(searchKey);

   if(delNode.leaf() == true) //need method to
       deleteNode(delNode);     //check if leaf

   else { //do the following with pred or succ
       AVLTreeNode temp = delNode.pred;
       deleteNode(delNode.pred);
       delNode.data = temp.data;
   }

} //end delete
```

AVL Tree DeleteNode

```
//private method of the AVLTree class
//Deletes the node delNode
void deleteNode(AVLTreeNode delNode) {

   //S contains nodes in the path from the root
     //to the node to be deleted
   Stack S = buildStack(delNode);

   AVLTreeNode child = S.pop();     //Initially,
   AVLTreeNode pivot = S.pop();     //child is node
   boolean     done  = false;       //to delete

   //continue to next page...
```

AVL Tree DeleteNode Case 1

```
while(!done) {
   //CASE 1: a node is removed from a perfectly
   //     balanced subtree; adjust bf and stop.
   if(pivot.bf == EQUAL_HT)
      if(child == pivot.right) {
         pivot.right = null;
         pivot.bf = -1;
      }
      else {
         pivot.left = null;
         pivot.bf = 1;
      }

      done = true;
   //continue to Case 2
```

AVL Tree DeleteNode Case 2

```
//CASE 2: a node is removed from the taller
//        subtree, pivot becomes perfectly
//        balanced; adjust bf and stop.
if((pivot.bf == TALL_RIGHT && child==pivot.right)
  ||(pivot.bf == TALL_LEFT && child ==pivot.left))
{
    if(delNode == child) {
        pivot.left = null; pivot.right = null; }
    pivot.bf = EQUAL_HT;

    if(pivot == root)
        done = true;
    else {
        child = pivot;
        pivot = S.pop(); }
} //continue to Case 3 & 4
```

AVL Tree DeleteNode Case 3 & 4

```
        //CASE 3: a node is removed from a shorter
        //      left subtree
        if(pivot.bf==TALL_RIGHT && child==pivot.left) {
            pivot.left = null;
            pivot.bf = 2;
            //Rotate RR or RL, then balance tree to root
            done = true;
        }
        //CASE 4: a node is removed from a shorter
        //      right subtree
        if((pivot.bf==TALL_LEFT && child==pivot.right) {
            pivot.right = null;
            pivot.bf = -2;
            //Rotate LL or LR, then balance tree to root
            done = true;
        }
    } //end outer while-loop
} //end delete method
```

Questions?